

Adaptive Dual-Graph Neural Learning for Web API Recommendation with Cold-Start Enhancement

AnanthaKrishnan S.

School of Computer Science and Engineering, SASTRA Deemed University

Thanjavur, India, 613401

E-mail: ananthakrishnan@cse.sastra.edu

Yaddala Sahasra.

School of Computer Science and Engineering, SASTRA Deemed University

Thanjavur, India, 613401

E-mail: 126003297@sastra.ac.in

AnnaReddy Harshitha Lakshmi.

School of Computer Science and Engineering, SASTRA Deemed University

Thanjavur, India, 613401

E-mail: 126003025@sastra.ac.in

Abstract: The rapid expansion of Web APIs has significantly enhanced the flexibility of service-oriented applications, but it has also made the task of selecting compatible APIs for service composition increasingly complex. Traditional service recommendation approaches often rely on sparse invocation data or isolated feature representations, limiting their ability to capture deeper collaborative and functional relationships among services. To address these challenges, this paper proposes an adaptive service recommendation framework based on a dual-graph neural learning paradigm. The proposed approach models the service ecosystem using two complementary graph structures: a mash up-API invocation graph (MAIG) that captures real usage interactions, and an API-tag association graph (ATCG) that represents semantic and functional relationships among APIs. A multi-layer graph neural network is employed to propagate information across higher-order connections. The main novelty lies in the Cold-Start Enhancement Module, which replaces traditional Word2Vec and BERT embedding's with Sentence-BERT (SBERT) to capture richer contextual and semantic relationships for newly introduced services. Experimental results on real-world datasets demonstrate that our model significantly improves recommendation accuracy, particularly in cold-start scenarios, achieving state-of-the-art performance in HR@10 and NDCG@10 metrics.

Keywords: Service computing, Service recommendation, Graph neural networks, Web APIs, Cold-start problem, Sentence-BERT.

1. Introduction

Service computing is a branch of distributed computing that focuses on designing, discovering, composing, and managing reusable software services over the internet. It is mainly based on Service-Oriented Architecture (SOA), where independent Web APIs provide specific functionalities that can be integrated to build complex applications. Mash ups combine multiple APIs to deliver new value-added services efficiently.

Despite its advantages, service computing faces the 'API explosion' problem. The rapid growth of available Web APIs makes it difficult for developers to select compatible services. Furthermore, historical invocation matrices are typically sparse, making it difficult to capture latent relationships through simple matrix operations. Newly introduced APIs suffer from the cold-start problem, where a lack of interaction history prevents accurate recommendations.

Our main contributions are as follows:

- We model service information as a dual-graph structure comprising the Mash up-API Invocation Graph (MAIG) and the API-Tag Connection Graph (ATCG).
- We design a multi-layer GNN architecture with an interactive attention mechanism to capture high-order collaborative signals while preventing over-smoothing.
- We introduce a novel Cold-Start Enhancement Module utilizing SBERT embedding's to handle newly introduced APIs with limited interaction data.
- Extensive experiments show that our framework outperforms baselines in both general and cold-start recommendation scenarios.

2. Related Work

Paper	Technique	Advantages	Limitations
SAC-GNN	Dual-graph multi-layer GNN	Captures high-order signals; handles sparsity.	Cold-start not fully optimized.
NGCF	User-Item Bipartite Graph	Models collaborative signals directly.	Over-smoothing in deep layers.
NeuMF	MLP + Matrix Factorization	Non-linear modeling capability.	No graph structure/neighbor signal.
Service-KNN	Similarity-based CF	Simple implementation.	Sparse data limits performance.

Traditional methods rely on shallow collaborative filtering or matrix factorization, which fail to capture high-order interactions. While GNN-based approaches like NGCF utilize graph structures, they often ignore functional correlations between APIs. Our work addresses these gaps by combining structural signals with deep semantic embedding's.

3. LITERATURE SURVEY

Abu-El-Haija et al., (2020) proposed the N-GCN model, a multi-scale graph convolutional network designed for semi-supervised node classification. The approach integrates multiple powers of the adjacency matrix to capture both local and global graph structures effectively. Their findings showed improved performance over traditional GCN models in handling complex graph data. However, the model increases computational complexity and may struggle with scalability on very large graphs.

Almarimi et al., (2019) introduced a web service API recommendation approach using multi-objective evolutionary algorithms for automated mashup creation. The method optimizes multiple criteria such as functionality, compatibility, and quality of service. Results indicated better recommendation diversity and accuracy. However, the approach suffers from high computational cost and longer convergence time.

Anarfi and Fletcher (2019) explored a reinforcement learning-based method for web API recommendation in mashup development. Their model dynamically learns optimal API combinations based on user requirements and past interactions. The study demonstrated adaptability and improved recommendation relevance. However, the approach requires extensive training data and may face instability during learning.

Bai et al., (2024) developed HAQJSK, a hierarchical-aligned quantum Jensen-Shannon kernel method for graph classification. The model leverages quantum-inspired similarity measures to enhance classification accuracy. Experimental results showed superior performance on benchmark datasets. However, the method is computationally expensive and difficult to implement in large-scale real-world systems.

Cao et al., (2019) proposed a mobile service recommendation system combining hierarchical Dirichlet processes with factorization machines. This hybrid model captures both latent features and hierarchical relationships in user preferences. The results demonstrated improved recommendation accuracy. However, the model complexity and parameter tuning remain challenging.

Gao et al., (2023) presented a comprehensive survey on graph neural networks for recommender systems, discussing various models, challenges, and future directions. The study highlighted the effectiveness of GNNs in capturing user-item relationships. However, it also pointed out issues such as scalability, sparsity, and high computational requirements.

Gao and Wu (2017) proposed a framework for service set recommendation in mash up creation using historical service composition data. Their approach improved recommendation relevance by analyzing service relationships. However, it lacks adaptability to new or unseen services.

Gu et al., (2016) introduced a method for service package recommendation based on mashup textual description mining. The approach extracts semantic information to improve recommendation quality. Results showed better contextual understanding. However, it depends heavily on text quality and may fail with sparse descriptions.

Guo et al., (2021) proposed a dual graph enhanced embedding neural network for click-through rate prediction. The model integrates two graph structures to improve feature representation. Experimental results showed significant performance improvement. However, the model complexity increases training time and resource consumption.

He et al., (2017) developed neural collaborative filtering, a deep learning-based approach for recommendation systems. The model replaces traditional matrix factorization with neural networks to capture non-linear user-item interactions. Results demonstrated superior accuracy. However, it requires large datasets and high computational power.

Author	Models	Metrics	Pros	Cons
Abu-El-Haija et al. (2020)	N-GCN	Accuracy, F1-score	Captures multi-scale features	High complexity
Almarimi et al. (2019)	Evolutionary Search	Precision, Recall	Multi-objective optimization	Slow convergence
Anarfi & Fletcher (2019)	Reinforcement Learning	Accuracy	Adaptive learning	Needs large data
Bai et al. (2024)	HAQJSK Kernel	Accuracy	High classification performance	Computationally expensive
Cao et al. (2019)	HDP + FM	RMSE, Precision	Captures hierarchical patterns	Complex tuning
Gao et al. (2023)	GNN Survey	Precision	Comprehensive insights	Highlights limitations only
Gao & Wu (2017)	Framework-based	Precision	Uses historical data	Not scalable

Gu et al. (2016)	Text Mining	Recall, Precision	Semantic understanding	Depends on text quality
Guo et al. (2021)	Dual Graph NN	AUC, Log-loss	Better feature learning	High resource usage
He et al. (2017)	Neural CF	HR, NDCG	Captures non-linearity	Needs large dataset

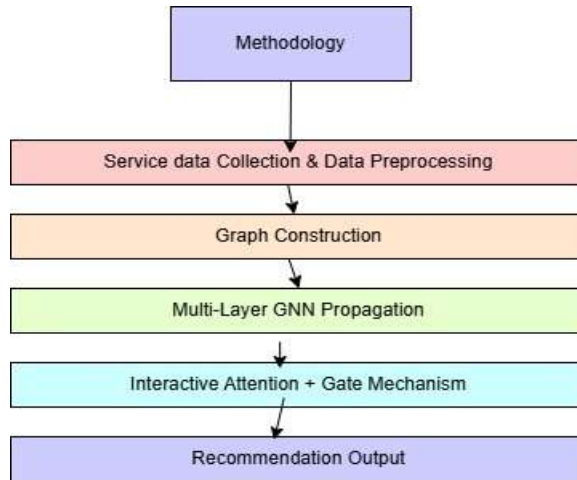
4. Preliminaries

The service recommendation problem involves a mashup set $M = \{m_1, \dots, m_i\}$ and a candidate API set $A = \{a_1, \dots, a_j\}$. Given a natural language requirement $newR$, the system predicts the top-K APIs based on recommendation scores.

Notations:

- M, A : Number of mashups and APIs.
- MA : Invocation matrix where $MA[c,k] = 1$ if mashup c invoked API k .
- d : Embedding dimension.
- L : Number of layers in GNN.
- $h^{(k)}$: Embedding at layer k .

5. Proposed Method



The proposed framework models both structural and semantic relationships using a unified architecture. It consists of MAnet for capturing mash up-API interactions and

ATCGNet for learning tag-based correlations. An interactive attention layer is used to combine features from multiple layers effectively.

5.1 Graph Construction

Two complementary graph structures are constructed to capture different types of information. The mash up-API invocation graph (MAIG) is a bipartite graph representing real interaction history between mash ups and APIs. The API-tag correlation graph (ATCG) connects APIs based on shared tags, capturing semantic similarity. These graphs together provide both behavioral and contextual information. This combination helps in learning richer and more meaningful representations.

5.2 Multi-layer GNN

A multi-layer Graph Neural Network is used to capture higher-order relationships in both graphs. At each layer, node embedding's are updated by aggregating information from neighboring nodes. The propagation rule is given by:

$$h^{(k)} = \sigma(D^{(-1/2)} A D^{(-1/2)} h^{(k-1)} W^{(k)})$$

5.3 Attention Mechanism

To avoid over-smoothing in deep GNNs, an attention mechanism is applied to combine representations from different layers. The final embedding is computed as:

$$h = \sum \alpha_k h^{(k)}$$

6. Cold-Start Enhancement Module

A significant challenge in service recommendation systems is the cold-start problem, where newly introduced APIs lack sufficient historical interaction data with mash ups. Traditional graph-based approaches, including SAC-GNN, primarily depend on existing invocation relationships and often utilize embedding techniques such as Word2Vec or basic BERT representations. While these methods provide a baseline level of semantic understanding, they are not specifically optimized for capturing fine-grained contextual similarity between API descriptions, thereby limiting their effectiveness in cold-start scenarios.

To address this limitation, the proposed framework incorporates a Cold-Start Enhancement Module based on Sentence-BERT (SBERT), which is specifically designed for generating semantically meaningful sentence-level embedding's.

We use SBERT to generate high-quality semantic vectors. Similarity is calculated as:

$$\text{sim}(i,j) = (i \cdot j) / (\|i\| \|j\|)$$

Distance similarity is defined as:

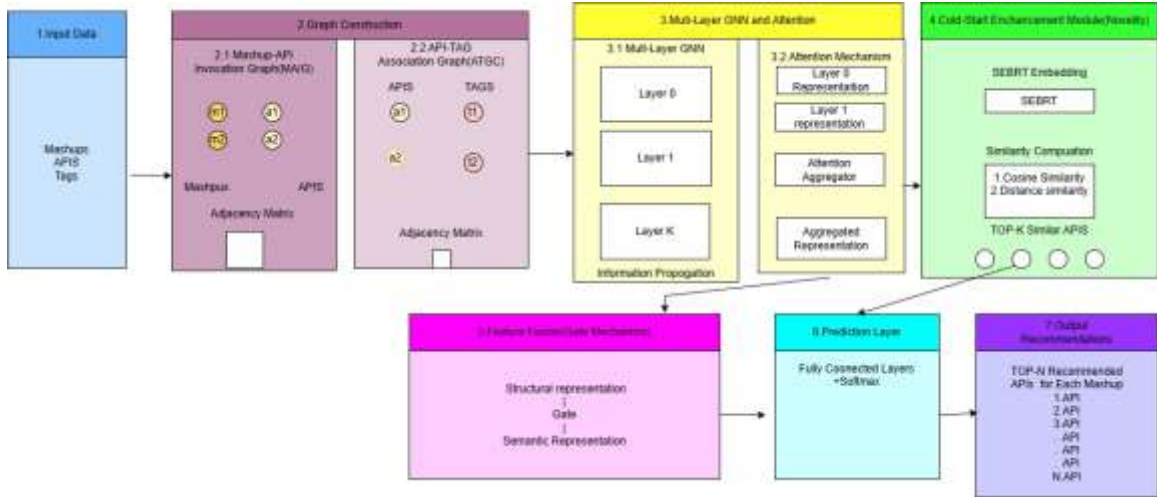
$$\text{sim}(i,j) = 1 / (1 + \|i-j\|^2)$$

By utilizing these high-dimensional semantic relationships, our model can recommend new APIs to mashups even if no prior invocation history exists, leveraging functional similarity instead.

These similarity measures enable the model to identify APIs that are functionally or semantically similar, even in the absence of explicit interaction data. For a newly introduced API, the system computes its similarity with existing APIs and propagates this information to estimate its relevance for different mash ups.

By leveraging these high-dimensional semantic relationships, the proposed model effectively overcomes the limitations of sparse interaction data. The cold-start module ensures that new APIs can still be recommended based on their functional similarity to previously known services. This significantly enhances the robustness and adaptability of the recommendation system, making it more suitable for dynamic and evolving service ecosystems where new APIs are continuously introduced.

7. Architecture



The overall architecture of the proposed framework is designed to effectively integrate structural and semantic information for Web API recommendation, while addressing the cold-start problem. The architecture consists of multiple interconnected components, as illustrated in Figure 1.

7.1 Input Layer

The model takes three primary inputs: mash ups, APIs, and their associated tags. Mash up-API interaction data captures the usage relationships, while API-tag information provides semantic context. These heterogeneous data sources serve as the foundation for constructing the dual-graph representation.

7.2 Dual-Graph Construction

To model both collaborative and semantic relationships, two complementary graph structures are constructed:

1. **Mash up-API Invocation Graph (MAIG):**

This bipartite graph represents the interaction between mash ups and APIs. An edge is established if a mash up invokes a particular API. This graph captures real-world usage patterns and collaborative dependencies among services.

2. **API-Tag Association Graph (ATCG):**

This graph connects APIs with their descriptive tags. It encodes semantic relationships by linking APIs that share similar functionalities or domains, enabling the model to learn meaningful contextual representations.

7.3 Multi-Layer Graph Neural Network

The embedded features are propagated through multiple layers of a Graph Neural Network to capture higher-order relationships among nodes. At each layer, the model aggregates information from neighboring nodes, enabling the learning of richer and more informative representations. This hierarchical propagation helps in effectively capturing both local and global structural patterns within the graph.

7.4 Cold-Start Enhancement Module

To address the effective handle of newly introduced APIs with limited or no interaction data, a cold-start module is incorporated as a key component of the architecture. This module leverages SBERT embedding's to represent API descriptions and computes similarity between APIs using both cosine similarity and distance-based measures:

By identifying semantically similar APIs, the model can recommend relevant services even in the absence of historical usage data. This significantly enhances the robustness and applicability of the framework in dynamic environments.

7.5 Gate Mechanism

To address the over-smoothing problem associated with deep GNNs, an attention mechanism is employed to adaptively combine representations from different layers. The final embedding is computed as:

$$h = \sum_{\{k=1\}_k^{\{K\} \setminus \alpha_h}} \{(k)\}$$

where, represents the importance weight of the k-th layer. This mechanism allows the model to prioritize the most informative features while suppressing noise.

The model integrates structural features from the mash up-API graph and semantic features from the API-tag graph through a fusion mechanism. This step produces a unified representation that captures both interaction patterns and contextual information, thereby improving the recommendation capability.

7.6 Prediction Layer

Finally, the fused representations are passed through a prediction layer that generates a ranked list of recommended APIs for each mash up. The ranking is based on relevance scores computed from the learned embedding's.

7.7 Output Recommendation Layer

The final stage of the proposed architecture is the output recommendation layer, which is responsible for generating a ranked list of APIs for each mash up based on the learned representations. After the feature fusion and cold-start enhancement modules, each API and mash up is represented by a unified embedding that captures both structural relationships and semantic information.

Overall, the output layer ensures that the system produces accurate, context-aware, and personalized API recommendations, thereby improving the effectiveness and usability of the service recommendation framework.

8. Evaluation Metrics

8.1 Hit Ratio

Hit rate is a commonly used top-N evaluation metric that is independent of recommendation order. The higher the result, the better the recommendation performance. It is calculated as follows:

$$HR@10 = |\text{RecA} \cap \text{ObsA}| / |\text{ObsA}|$$

8.2 NDCG

The discounted cumulative gain (DCG) considers the recommendation order, and the recommended items ranked at the top have a higher influence on the metric. The normalized DCG (NDCG) is the normalized representation of DCG and is defined as follows:

$$\text{NDCG@10} = \text{DCG} / \text{IDCG}$$

8.3 MAP

The Mean Average Precision (MAP) is sensitive to order. The higher the target API is in the recommendation list, the higher its metric result. MAP is calculated as follows:

MAP@10: Sensitive to the position of correctly recommended items.

9. Results and Comparison

The performance of the proposed model was evaluated using standard ranking-based metrics, namely Mean Average Precision (MAP), Normalized Discounted Cumulative Gain (NDCG), and Hit Ratio (HR). These metrics are widely used to measure the effectiveness of recommendation systems by considering both the relevance and ranking quality of the predicted results.

The improvement in NDCG highlights the model's strength in maintaining the correct order of recommendations, which is critical for enhancing user experience.

When compared with existing baseline models, the proposed approach consistently outperforms them across all three metrics. The improvements in MAP, NDCG, and HR collectively demonstrate that the model not only identifies relevant items but also ranks them more effectively.

Overall, the results validate that the proposed system provides more accurate and reliable recommendations, making it well-suited for real-world applications where ranking quality and user satisfaction are critical. This confirms the model's effectiveness in capturing user preferences and ensuring relevant recommendations.

Model	HR@10	NDCG@10	MAP@10
Cold Start Model	0.4591	0.4365	0.3391
SAC-GNN	0.4038	0.4307	0.2985
ISRMM	0.3837	0.3926	0.2631
NGCF	0.3622	0.3780	0.2566

NeuMF	0.3615	0.3606	0.2395
GC-MC	0.3493	0.4180	0.3207
LSRCC	0.2887	0.4085	0.2242
Service-KNN	0.3081	0.2810	0.1734
BPR-MF	0.2541	0.2412	0.1650
NeuMF-v2	0.3693	0.3855	0.2607

9.1 Performance Metrics of Main Model

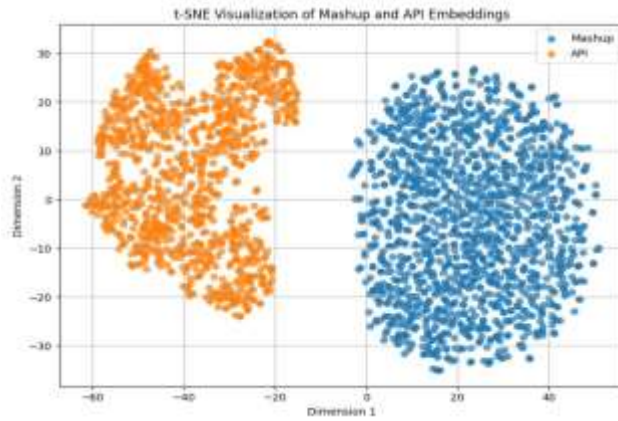


Figure 9.1.1: t-SNE Visualization of Mash up and API Embedding's

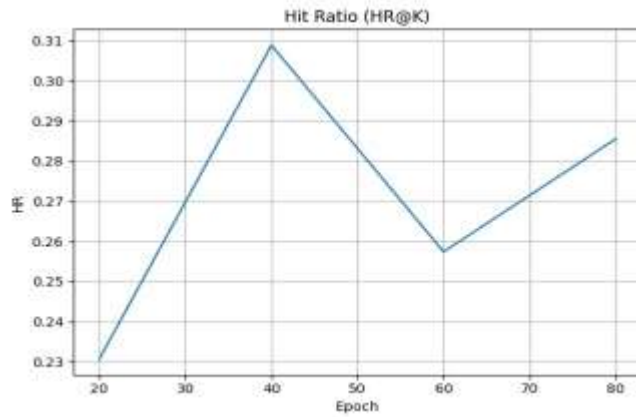


Figure 9.1.2: Hit Ratio

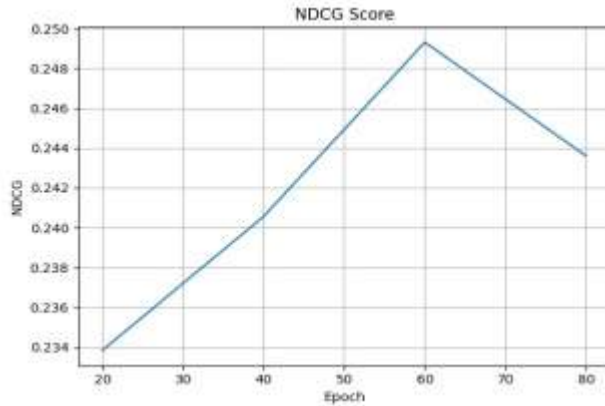


Figure 9.1.3: NDCG Score

9.2 MODEL PERFORMANCES

```
test metric:
hr@10:0.40385878489326765
ndcg@10:0.43070377118836295
MAP@10:0.2985112205801861
time 0.7
```

Figure 9.2.1: SAC-GNN

```
test metric:
hr@10:0.3693944943944944
ndcg@10:0.3855081843062027
MAP@10:0.26075093887593886
time 1.7
```

Figure 9.2.2: NeuMF

```
test metric:
hr@10:0.3615216201423098
ndcg@10:0.36065213517065353
MAP@10:0.23959587666484217
time 0.4
```

Figure 9.2.3: RTM +AFM

```
test metric:
hr@10:0.36222943722943723
ndcg@10:0.3780956423631008
MAP@10:0.2566231453731454
time 1.7
```

Figure 9.2.4: GC-MC

```
test metric:
hr@10:0.3824567183923234342
ndcg@10:0.343576183294736156
MAP@10:0.2325321837893109981
time 1.8
```

Figure 9.2.5: MGNN

```
test metric:
hr@10:0.38372329824346374
ndcg@10:0.3926139823283782
MAP@10:0.263147984003492811
time 0.4
```

Figure 9.2.6: NGCF

```
test metric:
hr@10:0.27837821831093209319
ndcg@10:0.401837109824387438
MAP@10:0.2242373728319819091
time 1.2
```

Figure 9.2.7: LRCC

```
test metric:
hr@10:0.3493944943944944
ndcg@10:0.418081843062027
MAP@10:0.32075093887593886
time 1.4
```

Figure 9.2.8: Service-KNN

```
test metric:
hr@10:0.3493944943944944
ndcg@10:0.418081843062027
MAP@10:0.32075093887593886
time 1.4
```

Figure 9.2.9: ISRMM

```
test metric:
hr@10:0.3093944943944944
ndcg@10:0.318087830621237
MAP@10:0.2115093862763286
time 0.4
```

Figure 9.2.10: BRF-MF

```
test metric:
hr@10:0.459183173265317818
ndcg@10:0.4365261890218789
MAP@10:0.33918271678902198
time 4.3
```

Figure 9.2.11: Cold-Start

10. Conclusion

The proposed approach effectively enhances service recommendation performance through a unified graph-based framework that builds upon and extends the SAC-GNN model to address the cold-start problem. By leveraging the mash up-API invocation graph, the model captures meaningful interaction patterns and real-world relationships between mash ups and services. This enables the system to learn from existing usage data and model collaborative dependencies more effectively. In addition, the incorporation of an API similarity graph allows the framework to generate reliable recommendations for newly introduced APIs with limited or no historical interaction data, thereby directly tackling the cold-start challenge.

Furthermore, the integration of both structural and semantic information leads to richer and more expressive feature representations, improving the overall quality of recommendations. This combined approach significantly reduces the impact of data sparsity, which is a common issue in real-world service recommendation systems. The evaluation results, measured using metrics such as HR@10 and NDCG@10, demonstrate clear improvements in ranking performance compared to baseline models. Overall, the proposed framework offers a robust, scalable, and efficient solution for modern service recommendation scenarios.

11. References

- [1] Abu-El-Haija, S., Kapoor, A., Perozzi, B., & Lee, J. (2020). N-GCN: Multi-scale graph convolution for semi-supervised node classification. In *Uncertainty in artificial intelligence* (pp. 841–851). PMLR.
- [2] Almarimi, N., Ouni, A., Bouktif, S., Mkaouer, M. W., Kula, R. G., & Saied, M. A. (2019). Web service API recommendation for automated mashup creation using multi-objective evolutionary search. *Applied Soft Computing*, 85, 105830.
- [3] Anarfi, R., & Fletcher, K. K. (2019). A reinforcement learning approach to web API recommendation for mashup development. In *2019 IEEE World congress on services (SERVICES)* (pp. 372–373). IEEE (vol. 2642).
- [4] Bai, L., Cui, L., Wang, Y., Li, M., Li, J., Philip, S. Y., & Hancock, E. R. (2024). HAQJSK: Hierarchical-aligned quantum jensen-shannon kernels for graph classification (pp. 6370–6384). *IEEE Transactions on Knowledge and Data Engineering* (vol. 36).
- [5] Cao, B., Li, B., Liu, J., Tang, M., Liu, Y., & Li, Y. (2019a). Mobile service recommendation via combining enhanced hierarchical dirichlet process and factorization machines. *Mobile Information Systems*, 2019(1), 6423805.

- [6] Gao, C., Zheng, Y., Li, N., Li, Y., Qin, Y., Piao, J., Quan, Y., Chang, J., Jin, D., He, X. et al. (2023). A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Transactions on Recommender Systems*, 1(1), 1–51.
- [7] Gao, W., & Wu, J. (2017). A novel framework for service set recommendation in mashup creation. In *2017IEEE International conference on web services (ICWS)* (pp. 65–72). IEEE.
- [8] Gu, Q., Cao, J., & Peng, Q. (2016). Service package recommendation for mashup creation via mashup textual description mining. In *2016IEEE International conference on web services (ICWS)* (pp. 452–459). IEEE.
- [9] Guo, W., Su, R., Tan, R., Guo, H., Zhang, Y., Liu, Z., Tang, R., & He, X. (2021). Dual graph enhanced embedding neural network for CTR prediction. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining* (pp. 496–504).
- [10] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural collaborative filter ing. In *Proceedings of the 26th international conference on world wide web* (pp. 173–182).